

Servicio Nacional de Aprendizaje SENA
Centro de Gestión de Mercados, Logística y Tecnologías de la Información
Tecnólogo en Análisis y Desarrollo de Software

GA5-220501095-AA1-EV06

**Nociones de Reglas de Usabilidad y Accesibilidad en Aplicaciones
Independientes Móviles**

Presentado por:

Gerson Jonnathan López Oviedo

Código de aprendiz: 3186647

Docente:

Arnaldo Alfonso Montiel Brun

Fase 5 Ejecución

Actividad de Proyecto 6

Bogotá D.C. 2025

Tabla de contenido

Introducción	3
Sección 1. Usabilidad en Aplicaciones Móviles	4
1.1 Definición y Componentes de la Usabilidad	4
1.2 Las 10 Heurísticas de Nielsen Aplicadas a TRUEQUE MATCH	5
Sección 2. Accesibilidad en Aplicaciones Móviles	6
2.1 Definición y Marco Normativo (WCAG 2.1 y Colombia)	6
2.2 Principios POUR Aplicados al Proyecto	7
Sección 3. App Independiente vs. App Móvil	9
Sección 4. App Nativa vs. App Híbrida	10
Sección 5. Ecosistemas de Dispositivos Móviles	12
Sección 6. Entornos de Desarrollo Integrado (IDE) para Móviles	14
Sección 7. Lenguajes de Programación para Dispositivos Móviles	15
Conclusiones	18
Referencias	19

Introducción

El presente documento corresponde a la evidencia GA5-220501095-AA1-EV06, desarrollada en el marco del Tecnólogo en Análisis y Desarrollo de Software del SENA. Su propósito es presentar los conceptos fundamentales de usabilidad y accesibilidad en aplicaciones móviles, establecer las diferencias entre los distintos tipos de aplicaciones existentes en el mercado y caracterizar los entornos de desarrollo, ecosistemas y lenguajes de programación utilizados en el desarrollo móvil actual.

Para entender de qué hablamos cuando decimos usabilidad, basta con pensar en cualquier aplicación que uses todos los días: WhatsApp, YouTube o el banco desde el celular. Si puedes hacer lo que necesitas sin frustrarte, sin perderte y sin necesitar un manual, esa aplicación tiene buena usabilidad. La accesibilidad, por su parte, va un paso más allá: garantiza que esa misma experiencia sea posible para personas con discapacidades visuales, auditivas o motoras. Como lo dice el principio del diseño universal, una buena app debe funcionar para todos, no solo para los que no tienen ninguna limitación.

Todos los conceptos desarrollados en este documento tienen aplicación directa en el proyecto TRUEQUE MATCH, la aplicación móvil de intercambio de bienes, servicios, conocimientos y experiencias que se ha venido construyendo a lo largo de las evidencias anteriores. Las decisiones de diseño ya tomadas en el prototipo, el mapa de navegación y la propuesta técnica se analizan aquí desde la perspectiva de la usabilidad, la accesibilidad y las mejores prácticas del desarrollo móvil.

La investigación se fundamenta en las heurísticas de Nielsen (1993), los principios de accesibilidad WCAG 2.1 del W3C (2018), la normativa colombiana aplicable (Ley 1618 de 2013) y referentes técnicos de Pressman y Maxim (2020), Google (2023) y Apple (2023) sobre diseño e ingeniería de aplicaciones móviles.

Sección 1. Usabilidad en Aplicaciones Móviles

1.1 Definición y Componentes de la Usabilidad

La usabilidad es, en términos simples, qué tan fácil y agradable resulta usar una aplicación. Nielsen (1993), quien es como el árbitro internacional de este tema, la define como el atributo de calidad que evalúa la facilidad con que los usuarios pueden emplear una interfaz. Pero no se queda ahí: la descompone en cinco componentes que funcionan como los cinco criterios de un buen árbitro para evaluar si una app está bien diseñada o no.

El primero es la capacidad de aprendizaje: ¿puede un usuario nuevo realizar tareas básicas la primera vez que abre la app sin que alguien le explique? El segundo es la eficiencia: una vez que el usuario ya conoce la app, ¿qué tan rápido puede hacer lo que necesita? El tercero es la memorabilidad: si alguien no usa la app por un mes y la vuelve a abrir, ¿puede retomar desde donde estaba sin empezar de cero? El cuarto es el manejo de errores: ¿cuántos errores comete el usuario, qué tan graves son y qué tan fácil es corregirlos? El quinto y último es la satisfacción subjetiva: ¿le gusta usar la app o le genera estrés?

En el contexto móvil, estos cinco componentes adquieren una dimensión adicional: la pantalla pequeña, el uso con los dedos (no con un ratón) y las condiciones variables de uso (en el bus, con una sola mano, con poca luz) hacen que diseñar una app móvil usable sea un reto más exigente que diseñar una aplicación de escritorio. Según Google (2023), el 53% de los usuarios móviles abandona una aplicación si tarda más de tres segundos en cargar, lo cual ilustra perfectamente la importancia de la eficiencia en el entorno móvil.

Para TRUEQUE MATCH, la usabilidad no es un elemento decorativo sino un requerimiento formal: el RNF05 establece explícitamente que la interfaz debe ser intuitiva y de fácil uso para personas con distintos niveles de alfabetización digital. Esto significa que tanto un universitario en Bogotá como un tendero en un municipio

rural de Colombia deben poder publicar una oferta o proponer un trueque sin necesitar capacitación previa.

1.2 Las 10 Heurísticas de Nielsen Aplicadas a TRUEQUE MATCH

la heurística en análisis y desarrollo web es una herramienta clave para evaluar y mejorar la usabilidad de un sitio, asegurando que sea claro, consistente y fácil de usar. de usabilidad de Nielsen (1994) son diez principios de diseño de interfaces que funcionan como una lista de chequeo para evaluar si una aplicación está bien pensada para el usuario. Son como las diez reglas de oro que todo diseñador debería tener en la pared. A continuación se presenta cada una con su aplicación concreta en TRUEQUE MATCH:

#	Heurística / Regla	¿Qué significa en la práctica?	Aplicación en TRUEQUE MATCH
1	Visibilidad del estado del sistema	El usuario siempre sabe qué está pasando. Como el marcador de un partido: nunca te deja sin información.	Barra de carga al publicar oferta; indicador de progreso en el registro (paso 1 de 3)
2	Coincidencia con el mundo real	El sistema habla el idioma del usuario, no el del programador. Nada de códigos raros ni tecnicismos innecesarios.	Textos como 'Publicar mi oferta', 'Ver mis trueques' en vez de 'POST /offers' o 'GET /swaps'
3	Control y libertad del usuario	El usuario puede deshacer acciones. Como el Ctrl+Z del computador: si te equivocas, puedes volver.	Botón de cancelar trueque activo; opción de editar oferta antes de publicar definitivamente
4	Consistencia y estándares	Los botones, colores y palabras significan lo mismo en toda la app. No cambies las reglas del juego a mitad de partido.	Paleta rojo/negro/blanco en todas las pantallas; botón rojo siempre = acción principal
5	Prevención de errores	Mejor evitar el error que tener que corregirlo. Como los topes en el parqueadero: no dejan que te pases.	Validación en tiempo real en formularios de registro y publicación de ofertas
6	Reconocimiento sobre recuerdo	El usuario no debería tener que memorizar cómo usar la app. Todo debe estar visible.	Menú inferior siempre visible con íconos reconocibles (casa=inicio, lupa=buscar, + =publicar)

7	Flexibilidad y eficiencia	Los usuarios expertos pueden ir más rápido sin molestar a los novatos.	Búsqueda por categorías (rápida) y filtros avanzados (para usuarios frecuentes)
8	Diseño estético y minimalista	Menos es más. Cada elemento en pantalla debe estar ahí por una razón.	Pantallas limpias con máximo 2-3 acciones por vista; sin menús recargados
9	Ayuda a reconocer y recuperarse de errores	Los mensajes de error deben decir exactamente qué pasó y cómo solucionarlo.	Pantalla 4 del prototipo: mensajes específicos por campo con color rojo en el error y verde al corregir
10	Ayuda y documentación	Si el usuario necesita ayuda, que sea fácil de encontrar y directa al grano.	Sección de ayuda contextual y tooltips en formularios complejos como proponer trueque

Nota. Las 10 heurísticas de Nielsen (1994) aplicadas al diseño de TRUEQUE MATCH.

Sección 2. Accesibilidad en Aplicaciones Móviles

2.1 Definición y Marco Normativo (WCAG 2.1 y Colombia)

La accesibilidad en software es la capacidad de una aplicación para ser utilizada por el mayor número posible de personas, incluyendo aquellas con discapacidades visuales, auditivas, motoras o cognitivas. Si la usabilidad se pregunta '¿qué tan fácil es usar esto?', la accesibilidad se pregunta '¿puede usarlo todo el mundo?'. Son como dos hermanos: van de la mano pero no son lo mismo.

El estándar internacional de referencia en accesibilidad web y móvil es la WCAG 2.1 (Web Content Accessibility Guidelines), publicada por el Consorcio World Wide Web (W3C, 2018). Esta guía define tres niveles de conformidad: nivel A (mínimo obligatorio), nivel AA (estándar recomendado para la mayoría de aplicaciones) y nivel AAA (máximo, generalmente solo para contextos especializados). Para TRUEQUE MATCH se estableció el nivel AA como objetivo de accesibilidad, que es el nivel exigido en la mayoría de legislaciones y políticas gubernamentales de los países hispanohablantes.

En Colombia, la Ley 1618 de 2013 establece la obligación de garantizar el pleno ejercicio de los derechos de las personas con discapacidad, lo que incluye el acceso en igualdad de condiciones a las tecnologías de la información. Complementariamente, la Resolución 1519 de 2020 del MinTIC establece los estándares de accesibilidad que deben cumplir los sitios web y aplicaciones de las entidades públicas colombianas, tomando como referencia precisamente la WCAG 2.1 en nivel AA. Si bien TRUEQUE MATCH es un proyecto privado, adoptar estos estándares desde el diseño inicial es una decisión técnica responsable y diferenciadora.

2.2 Principios POUR Aplicados al Proyecto

Los principios POUR son el corazón de la WCAG 2.1. Su nombre es un acrónimo de las cuatro condiciones que toda interfaz accesible debe cumplir: Perceptible, Operable, Comprensible y Robusto (W3C, 2018). Piénsalos como los cuatro pilares de un puente: si falla uno, el puente entero se cae. A continuación se presenta cada principio con su aplicación en TRUEQUE MATCH:

Principio WCAG 2.1	Descripción práctica	Aplicación en TRUEQUE MATCH
Perceptible	La información debe poder percibirse por al menos un sentido. Si es visual, también debe ser audible o táctil.	Textos alternativos en imágenes de ofertas; contraste de color 4.5:1 mínimo en toda la interfaz
Operable	Los elementos de interacción deben poder usarse con cualquier dispositivo de entrada.	Tamaño mínimo de botones táctiles de 48x48 píxeles; navegación por gestos y por voz
Comprensible	La información y el funcionamiento de la interfaz deben ser entendibles.	Etiquetas claras en todos los campos; mensajes de error descriptivos y sin tecnicismos

Robusto	El contenido debe funcionar correctamente con tecnologías de asistencia.	Compatibilidad con TalkBack (Android) y VoiceOver (iOS) para usuarios con discapacidad visual
----------------	--	---

*Nota. Principios POUR de la WCAG 2.1 aplicados al diseño de TRUEQUE MATCH.
Elaboración propia con base en W3C (2018) y la Ley 1618 de 2013.*

Sección 3. App Independiente vs. App Móvil

Una de las primeras distinciones que debe manejar un desarrollador de software es la diferencia entre una aplicación independiente (también llamada de escritorio o desktop) y una aplicación móvil. Aunque ambas son software, están diseñadas para contextos de uso, dispositivos y condiciones completamente distintas.

Imagina que estás en una oficina con tu computador, sentado cómodamente, con una pantalla grande, un teclado y un ratón: ese es el mundo de las aplicaciones independientes. Ahora imagina que estás en el bus de TransMilenio, parado, con el celular en una mano y el tote en la otra, con el sol dando en la pantalla: ese es el mundo de las aplicaciones móviles. El mismo objetivo (que puedas revisar tu cuenta bancaria, por ejemplo) requiere soluciones de diseño completamente diferentes según en cuál de esos dos mundos estés.

Las aplicaciones independientes se ejecutan directamente sobre el sistema operativo de un computador (Windows, macOS o Linux), tienen acceso completo a los recursos del hardware como procesador, memoria RAM y almacenamiento, y generalmente se distribuyen mediante instaladores descargables desde sitios web o tiendas especializadas. Las aplicaciones móviles, en cambio, se ejecutan sobre sistemas operativos diseñados específicamente para dispositivos portátiles con pantalla táctil, recursos limitados y conectividad variable, y se distribuyen principalmente a través de tiendas de aplicaciones controladas por los fabricantes del sistema operativo.

Para TRUEQUE MATCH, esta distinción tiene implicaciones directas: la versión principal del sistema es una aplicación móvil en React Native para Android e iOS, dado que el intercambio de objetos y servicios es una actividad que los usuarios realizan en movimiento y desde cualquier lugar. El panel de administración, en cambio, podría implementarse como una aplicación web (que comparte características con las aplicaciones independientes) dado que los administradores

trabajan típicamente desde un computador fijo en un entorno controlado. La siguiente tabla sintetiza las diferencias más relevantes:

Característica	App Independiente (Desktop)	App Móvil
Plataforma	Windows, macOS, Linux	Android, iOS
Tipo de dispositivo	PC / portátil / servidor	Smartphone / tablet / wearable
Interacción del usuario	Teclado y ratón principalmente	Pantalla táctil, voz, giroscopio
Tamaño de pantalla	Grande (13" a 27" o más)	Pequeño (4" a 7" típicamente)
Conectividad	Ethernet / Wi-Fi estable	Wi-Fi / datos móviles variables
Recursos disponibles	CPU y RAM amplios	CPU, RAM y batería limitados
Distribución	Descarga directa / instalador	App Store / Google Play
Actualización	Manual o automática por el SO	Automática vía tienda de apps
Ejemplo en el proyecto	Panel admin en navegador web	App TRUEQUE MATCH en Android

Nota. Diferencias entre aplicaciones independientes y aplicaciones móviles aplicadas al contexto de TRUEQUE MATCH. Elaboración propia con base en Pressman y Maxim (2020).

Sección 4. App Nativa vs. App Híbrida

Dentro del mundo de las aplicaciones móviles existe una segunda gran distinción que define el enfoque tecnológico del proyecto: si la aplicación va a ser nativa o híbrida. Esta decisión es tan importante como elegir entre construir una casa

en madera o en concreto: ambas te dan techo, pero el proceso, los materiales y el resultado son diferentes.

Una aplicación nativa es aquella que se desarrolla exclusivamente para un sistema operativo específico utilizando los lenguajes y herramientas oficiales de ese ecosistema. Es como contratar un cocinero que solo sabe hacer comida italiana: va a ser el mejor haciendo pasta, pero si quieres sushi tienes que contratar otro. Para Android, el lenguaje nativo oficial es Kotlin (anteriormente Java); para iOS, es Swift (anteriormente Objective-C). El resultado es una aplicación que aprovecha al máximo las capacidades del hardware del dispositivo y se integra perfectamente con los elementos de la interfaz del sistema operativo, brindando la mejor experiencia posible al usuario de ese ecosistema.

Una aplicación híbrida, en cambio, se desarrolla con tecnologías que funcionan en múltiples sistemas operativos desde un único código fuente. Es como un cocinero que sabe hacer comida de todos los países: no será el mejor en ninguno, pero puede servirte en todos. Frameworks como React Native, Flutter o Ionic permiten escribir el código una sola vez y ejecutarlo tanto en Android como en iOS, con resultados que en la mayoría de los casos son prácticamente indistinguibles de las apps nativas para el usuario final. Esta es la razón por la que TRUEQUE MATCH adopta React Native como tecnología de desarrollo móvil: un solo equipo, un solo repositorio de código y entrega simultánea en ambas plataformas.

Criterio	App Nativa	App Híbrida
Definición	Desarrollada específicamente para un SO (Android o iOS)	Un solo código base que funciona en varios SO
Lenguajes típicos	Kotlin/Java (Android) — Swift/Obj-C (iOS)	React Native, Flutter, Ionic, Xamarin
Rendimiento	Máximo rendimiento, acceso total al hardware	Rendimiento bueno, pero con pequeña capa intermedia

Acceso al hardware	Total: cámara, GPS, NFC, sensores, etc.	Casi total mediante plugins o puentes nativos
Costo de desarrollo	Alto: se necesitan equipos separados por SO	Menor: un solo equipo para todos los SO
Experiencia de usuario	UX óptima, sigue guías del SO exactamente	UX muy buena, puede diferir levemente entre SO
Velocidad de desarrollo	Más lenta por código separado	Más rápida por código compartido
Mantenimiento	Doble esfuerzo (Android + iOS)	Un solo repositorio para todos los SO
Ejemplo real	App de Instagram nativa en iOS	App de Facebook con React Native
TRUEQUE MATCH usa	No aplica para la primera fase	React Native: un código para Android e iOS

Nota. Comparación entre aplicaciones móviles nativas e híbridas. Elaboración propia con base en Pressman y Maxim (2020) y la propuesta técnica de TRUEQUE MATCH (López Oviedo, 2025).

Sección 5. Ecosistemas de Dispositivos Móviles

Un ecosistema móvil es el conjunto integrado de hardware, sistema operativo, tienda de aplicaciones, servicios en la nube y herramientas de desarrollo que una empresa construye alrededor de sus dispositivos. Es como un estadio completo: no solo el campo de juego (el celular), sino las tribunas (la tienda de apps), los camerinos (el sistema operativo), los árbitros (las reglas y políticas del fabricante) y los restaurantes dentro del estadio (los servicios complementarios). Los cuatro ecosistemas más relevantes en el mercado global actual son los siguientes:

Ecosistema	Empresa creadora	Cuota de mercado	Tienda de apps	IDE principal
Android	Google (Alphabet Inc.)	~72% global (StatCounter, 2024)	Google Play Store	Android Studio
iOS	Apple Inc.	~27% global (StatCounter, 2024)	App Store	Xcode
HarmonyOS	Huawei Technologies	~2-4% (creciendo en Asia)	Huawei AppGallery	DevEco Studio
KaiOS	KaiOS Technologies	<1% (mercados emergentes)	KaiStore	Web APIs estándar

Nota. Principales ecosistemas de dispositivos móviles a nivel global. Datos de participación de mercado tomados de StatCounter (2024). Elaboración propia (2025).

Para el contexto colombiano, los datos de StatCounter (2024) indican que Android domina con aproximadamente el 80% del mercado de dispositivos móviles en el país, seguido de iOS con cerca del 19%. Esta distribución es consistente con los mercados latinoamericanos en general, donde el menor precio promedio de los dispositivos Android los hace más accesibles para la mayoría de la población. Esta realidad refuerza la decisión técnica de TRUEQUE MATCH de priorizar el desarrollo en Android para la primera fase del proyecto, sin descuidar la compatibilidad con iOS gracias a React Native.

El ecosistema de Android se destaca por su naturaleza abierta: Google permite a los fabricantes de dispositivos (Samsung, Xiaomi, Motorola, Huawei, etc.) personalizar el sistema operativo con sus propias capas de interfaz, lo que genera una gran variedad de tamaños de pantalla, resoluciones y versiones del sistema operativo que el desarrollador debe tener en cuenta al diseñar y probar su aplicación.

iOS, en contraste, solo funciona en dispositivos Apple, lo que genera un ecosistema más controlado y predecible pero con menos variedad de hardware. Para TRUEQUE MATCH, esto significa que las pruebas de compatibilidad en Android deben realizarse en un conjunto representativo de dispositivos de distintas gamas y fabricantes.

Sección 6. Entornos de Desarrollo Integrado (IDE) para Móviles

Un IDE (Integrated Development Environment, o Entorno de Desarrollo Integrado en español) es el programa que usa el desarrollador para escribir, depurar, compilar y probar su código. Es como el taller completo del mecánico: no solo tiene la llave inglesa (el editor de texto), sino también el elevador (el emulador del dispositivo), el diagnóstico electrónico (el depurador) y el manual del motor (la documentación integrada). Sin un buen IDE, desarrollar una aplicación móvil sería como intentar construir un edificio con solo un martillo.

Los IDEs para desarrollo móvil se pueden clasificar en dos categorías: los IDEs específicos de plataforma, que están optimizados para un solo ecosistema (como Android Studio para Android o Xcode para iOS), y los IDEs multiplataforma, que soportan varios lenguajes y frameworks y son preferidos para el desarrollo híbrido. A continuación se presentan los principales IDEs del mercado y su relevancia para el proyecto TRUEQUE MATCH:

IDE	Desarrollador	Plataforma objetivo	Lenguajes soportados	Licencia
Android Studio	Google	Android	Kotlin, Java	Gratuita (Apache 2.0)
Xcode	Apple	iOS / macOS	Swift, Objective-C	Gratuita (solo macOS)
Visual Studio Code	Microsoft	Multiplataforma	JS, TS, Dart, Python, C#...	Gratuita (MIT)

IntelliJ IDEA	JetBrains	Multiplataforma	Kotlin, Java, Groovy...	Comunidad: gratis / Ultimate: pago
VS Code + Expo	Microsoft / Expo	Android e iOS	JavaScript, TypeScript	Gratuita
DevEco Studio	Huawei	HarmonyOS	Java, JavaScript, C++	Gratuita
Fleet	JetBrains	Multiplataforma	Kotlin, Swift, JS, Python...	Pago (en desarrollo)

Nota. Principales IDEs para el desarrollo de aplicaciones móviles. Elaboración propia con base en JetBrains (2023), Google (2023) y Apple (2023).

Para el desarrollo de TRUEQUE MATCH se seleccionó Visual Studio Code como IDE principal, en combinación con Expo (plataforma de herramientas que simplifica el desarrollo con React Native). Esta elección se fundamenta en tres criterios: primero, VS Code es gratuito y de código abierto, lo que se alinea con la propuesta de costos reales del proyecto que prioriza herramientas sin licencia. Segundo, cuenta con una extensión oficial de React Native que integra el depurador, el emulador y las herramientas de análisis de código directamente en el editor. Tercero, es el IDE más utilizado por la comunidad de desarrolladores JavaScript a nivel mundial según la encuesta anual de Stack Overflow (2023), lo que garantiza una amplia disponibilidad de recursos de aprendizaje, extensiones y soporte comunitario.

Sección 7. Lenguajes de Programación para Dispositivos Móviles

Si el IDE es el taller del mecánico, el lenguaje de programación es el idioma en que se comunica con el motor del auto. Elegir el lenguaje correcto no es solo una cuestión técnica: impacta directamente en el rendimiento de la aplicación, el tamaño del equipo de desarrollo requerido, el tiempo de entrega y los costos del proyecto. En el desarrollo móvil existen dos grandes familias de lenguajes: los nativos (específicos

para un sistema operativo) y los de propósito general con frameworks multiplataforma.

Cada lenguaje tiene sus fortalezas, su comunidad y su ecosistema de herramientas. Elegir mal puede ser costoso: es como construir tu casa en una ciudad sin saber que el terreno es inundable. Por eso, en la propuesta técnica de TRUEQUE MATCH se realizó un análisis comparativo de las opciones disponibles antes de seleccionar el stack tecnológico definitivo. A continuación se presenta un resumen de los principales lenguajes del ecosistema móvil y su relación con el proyecto:

Lenguaje	Tipo de app	SO objetivo	Ventajas principales	Uso en TRUEQUE MATCH
Kotlin	Nativa	Android	Moderno, seguro, conciso. Reemplaza a Java como estándar oficial de Google	No aplica en fase 1 (se usa RN)
Java	Nativa	Android	Maduro, amplia comunidad y recursos de aprendizaje	No aplica en fase 1 (se usa RN)
Swift	Nativa	iOS / macOS	Alto rendimiento, sintaxis clara, creado por Apple	Considerado para fase 2 (iOS nativa)
Objective-C	Nativa	iOS / macOS	Legacy, aún presente en proyectos antiguos de Apple	No aplica en el proyecto
JavaScript / TypeScript	Híbrida	Android, iOS, Web	Un solo lenguaje para frontend, backend y móvil. Alta productividad	Lenguaje principal: Node.js (backend) y React Native (móvil)
Dart	Híbrida	Android, iOS, Web, Desktop	Alto rendimiento en apps híbridas gracias a Flutter	Alternativa evaluada; se descartó en favor de React Native
C# / .NET MAUI	Híbrida	Android, iOS, Windows	Potente para empresas con ecosistema Microsoft	No aplica en el proyecto
HTML5 + CSS3 + JS	Web / PWA	Cualquier SO con navegador	Sin instalación, multiplataforma total	Versión web de TRUEQUE MATCH en fases futuras

Nota. Lenguajes de programación para el desarrollo de aplicaciones móviles. Elaboración propia con base en Pressman y Maxim (2020) y la propuesta técnica de TRUEQUE MATCH (López Oviedo, 2025).

La elección de JavaScript y TypeScript como lenguajes principales de TRUEQUE MATCH responde a una estrategia de unificación tecnológica: el mismo lenguaje que se usa para el frontend móvil (React Native) es el que se usa para el backend (Node.js con Express) y eventualmente para la versión web (React.js). Esto significa que un desarrollador del equipo puede trabajar en cualquier capa del sistema sin necesidad de aprender un lenguaje diferente, lo que reduce los costos de formación y facilita la rotación de tareas dentro del equipo. TypeScript, que es una versión de JavaScript con tipado estático, se añade para reducir los errores en tiempo de desarrollo y mejorar la calidad del código, en línea con los principios de calidad de software establecidos en la norma ISO/IEC 25010 (2011).

Conclusiones

El desarrollo de esta evidencia permitió comprender de manera integral los conceptos de usabilidad y accesibilidad como pilares del diseño de aplicaciones móviles de calidad, y establecer con claridad las diferencias entre los distintos tipos de aplicaciones, ecosistemas, IDEs y lenguajes de programación disponibles en el mercado actual.

La usabilidad, analizada desde las diez heurísticas de Nielsen (1994), no es un elemento opcional del diseño de TRUEQUE MATCH sino un requerimiento funcional que determina si los usuarios adoptarán o abandonarán la aplicación. Las decisiones de diseño ya tomadas en el prototipo (menú inferior persistente, validación en tiempo real, mensajes de error contextuales, paleta de colores consistente) responden directamente a estos principios y pueden ahora justificarse con fundamento técnico y normativo.

La accesibilidad, enmarcada en los principios POUR de la WCAG 2.1 (W3C, 2018) y la Ley 1618 de 2013 de Colombia, añade una dimensión de responsabilidad social al proyecto: TRUEQUE MATCH no solo debe ser fácil de usar para el usuario promedio, sino que debe garantizar que las personas con discapacidad visual, auditiva o motora puedan participar en la economía del intercambio en igualdad de condiciones.

Finalmente, la selección de React Native como framework híbrido, Visual Studio Code como IDE y JavaScript/TypeScript como lenguajes principales se sustenta en criterios técnicos, económicos y estratégicos coherentes con los requerimientos del proyecto y con las mejores prácticas de la industria del desarrollo móvil en el contexto colombiano y latinoamericano.

Referencias

Apple Inc. (2023). *Human interface guidelines for iOS*.

<https://developer.apple.com/design/human-interface-guidelines>

Google. (2023). *Material Design 3: Foundations*. <https://m3.material.io/foundations>

International Organization for Standardization. (2011). *ISO/IEC 25010: Systems and software quality requirements and evaluation System and software quality models*. ISO.

JetBrains. (2023). *Developer ecosystem survey 2023*.

<https://www.jetbrains.com/lp/devecosystem-2023>

López Oviedo, G. J. (2025). *Propuesta técnica y económica TRUEQUE MATCH*.

G.L. Software Solutions. SENA.

Nielsen, J. (1993). *Usability engineering*. Academic Press.

Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. En

Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (pp. 152-158). ACM. <https://doi.org/10.1145/191666.191729>

Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.^a ed.). McGraw-Hill Education.

República de Colombia. (2013). *Ley 1618 de 2013: Por medio de la cual se establecen las disposiciones para garantizar el pleno ejercicio de los derechos de las personas con discapacidad*. Congreso de la República.

República de Colombia. (2020). *Resolución 1519 de 2020: Por la cual se definen los estándares y lineamientos de accesibilidad digital*. Ministerio de Tecnologías de la Información y las Comunicaciones.

Stack Overflow. (2023). *Developer survey 2023*.
<https://survey.stackoverflow.co/2023>

StatCounter. (2024). *Mobile operating system market share Colombia — December 2024*. <https://gs.statcounter.com/os-market-share/mobile/colombia>

W3C. (2018). *Web content accessibility guidelines (WCAG) 2.1*.
<https://www.w3.org/TR/WCAG21>